ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

ОРГАНИЗАЦИЯ ПЕРЕДАЧИ УПРАВЛЯЮЩИХ КОМАНД НА УДАЛЁННОЕ ОБОРУДОВАНИЕ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков организации передачи управляющих команд на удаленное оборудование.

2 Задание и исходные данные

Организовать передачу управляющих команд из интерфейса на оборудование для всех вещей. Передачу управляющих команд можно реализовать как при помощи нескольких запросов, по одному для каждой вещи, так и одним запросом, который будет передавать все команды, а их обработкой будет заниматься отдельная функция на сервере. В рамках данной лабораторной работы необходимо реализовать передачу именно управляющих команд, переделывать запросы из прошлой лабораторной не нужно.

3 Ход выполнения задания

Для каждой вещи добавлен метод connect_with_commands(request): из request.args читается параметр value и обновляются поля объекта. На странице эмулятора добавлены поля ввода и кнопки, в index.js — функции отправки ajax с data. Результаты работы представлены на рисунке 1 – 2.

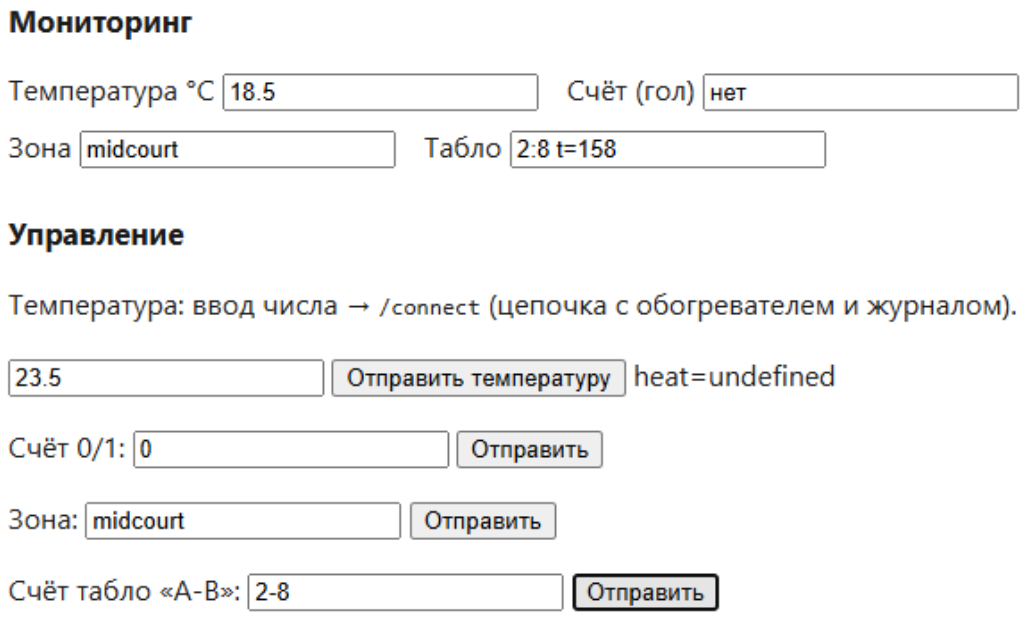

Рисунок 1 – Результаты работы программы

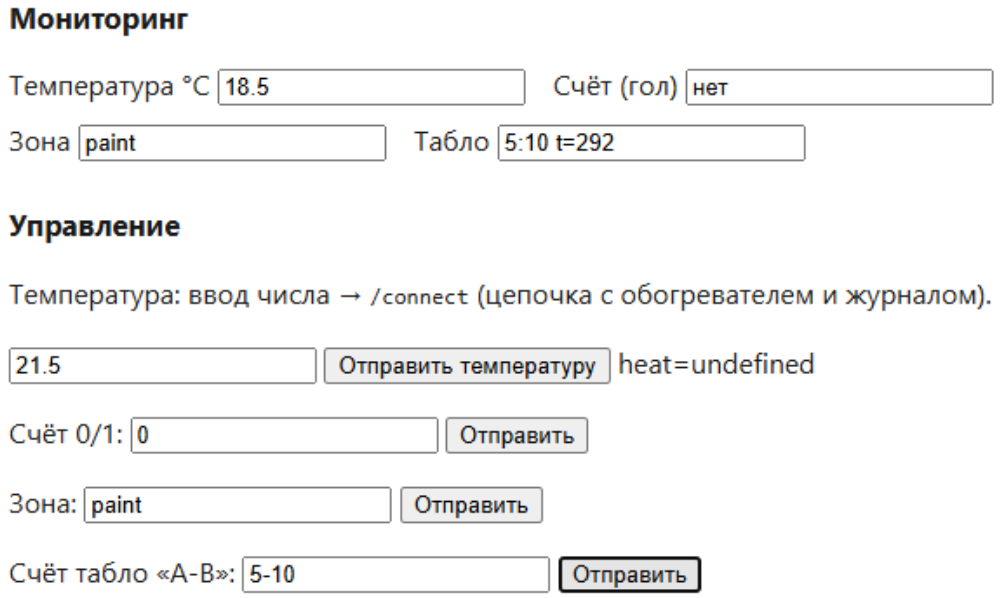

Рисунок 2 – Результаты работы программы

4 Вывод

В ходе выполнения лабораторной работы была успешно организована передача управляющих команд из веб-интерфейса на удаленное оборудование. Для каждого класса вещей (ScoreSensor, ZoneSensor, EnvSensor, Scoreboard) был добавлен метод connect_with_commands(request), который извлекает параметр value из request.args и обновляет соответствующие поля объекта, имитируя передачу команды на физическое устройство.

На серверной стороне (app.py) были созданы маршруты для обработки управляющих команд, дополняющие существующую систему мониторинга. В шаблон sensor_emulator.html были добавлены элементы управления: поля ввода и кнопки для каждой вещи, позволяющие пользователю задавать новые значения температуры, счета, зоны и показаний табло.

Таким образом, цель работы по приобретению навыков организации передачи управляющих команд на удаленное оборудование достигнута в полном объеме.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
// ============================
// File: app.py
// ============================
from __future__ import annotations

from flask import Flask, jsonify, render_template

import things

app = Flask(__name__)

env_sensor = things.EnvSensor("es1", "Микроклимат зала", 22.0, 400.0)
score_sensor = things.ScoreSensor("ss1", "Кольцо команды A", is_goal=False)
zone_sensor = things.ZoneSensor("zs1", "Разметка площадки", "three-point-arc")
scoreboard = things.Scoreboard("sb1", "Табло арены", 0, 0, 0)

store = things.CourtDataStore()
mcu = things.MainControlUnit(
    [score_sensor, zone_sensor, env_sensor, scoreboard],
    store,
    scoreboard,
)


def demo_iot_cycle() -> None:
    print("=== collectData ===")
    mcu.collectData()
    print("=== calculateScore ===")
```

```
    mcu.calculateScore()
    print("=== updateScoreboard ===")
    mcu.updateScoreboard()
    print("=== getRecords ===", len(store.getRecords()))


@app.route("/monitor/env")
def monitor_env():
    return jsonify(env_sensor.connect())


@app.route("/monitor/score")
def monitor_score():
    return jsonify(score_sensorfrom __future__ import annotations

from flask import Flask, jsonify, render_template, request

import things

app = Flask(__name__)

env_sensor = things.EnvSensor("es1", "Микроклимат зала", 22.0, 400.0)
score_sensor = things.ScoreSensor("ss1", "Кольцо команды A", is_goal=False)
zone_sensor = things.ZoneSensor("zs1", "Разметка площадки", "three-point-arc")
scoreboard = things.Scoreboard("sb1", "Табло арены", 0, 0, 0)

store = things.CourtDataStore()
mcu = things.MainControlUnit(
    [score_sensor, zone_sensor, env_sensor, scoreboard],
    store,
```

```
    scoreboard,
)

def demo_iot_cycle() -> None:
    print("=== collectData ===")
    mcu.collectData()
    print("=== calculateScore ===")
    mcu.calculateScore()
    print("=== updateScoreboard ===")
    mcu.updateScoreboard()
    print("=== getRecords ===", len(store.getRecords()))

@app.route("/monitor/env")
def monitor_env():
    return jsonify(env_sensor.connect())

@app.route("/monitor/score")
def monitor_score():
    return jsonify(score_sensor.connect())

@app.route("/monitor/zone")
def monitor_zone():
    return jsonify(zone_sensor.connect())

@app.route("/monitor/board")
def monitor_board():
    return jsonify(scoreboard.connect())

@app.route("/connect")
def connect():
```

```
    return jsonify(env_sensor.connect_with_commands(request))

@app.route("/command/score")
def command_score():
    return jsonify(score_sensor.connect_with_commands(request))

@app.route("/command/zone")
def command_zone():
    return jsonify(zone_sensor.connect_with_commands(request))

@app.route("/command/board")
def command_board():
    return jsonify(scoreboard.connect_with_commands(request))

@app.route("/lab2")
def lab2_page():
    demo_iot_cycle()
    return render_template("lab2.html")

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/emulator")
def emulator_page():
    return render_template("sensor_emulator.html")

if __name__ == "__main__":
    app.run(debug=True)
```

```
// ============================
// File: things.py
// ============================
from __future__ import annotations

import abc
import random
from typing import Any, Dict, List, Optional

class Thing(abc.ABC):

    def __init__(self, thing_id: str, name: str) -> None:
        self.id = thing_id
        self.name = name

    @abc.abstractmethod
    def getStatus(self) -> str:
        pass

    def emulate(self) -> None:
        pass

    def connect(self, request: Any = None) -> Dict[str, Any]:
        raise NotImplementedError

class ScoreSensor(Thing):
    def __init__(self, thing_id: str, name: str, is_goal: bool = False) -> None:
        super().__init__(thing_id, name)
        self.is_goal = is_goal
```

```
    def getStatus(self) -> str:
        return "ScoreSensor[" + self.id + "]: is_goal=" + str(self.is_goal)

    def emulate(self) -> None:
        self.is_goal = random.choice([True, False])

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {"id": self.id, "value": 1 if self.is_goal else 0, "is_goal": self.is_goal}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        """Лаб. 4: команда из request.args (методичка, рис. 22)."""
        if request is not None:
            raw = request.args.get("value", "")
            if raw != "":
                self.is_goal = bool(int(float(raw)))
        return {"power": "on", "is_goal": self.is_goal}

class ZoneSensor(Thing):
    ZONES = ("three-point-arc", "paint", "midcourt", "baseline")

    def __init__(self, thing_id: str, name: str, zone: str) -> None:
        super().__init__(thing_id, name)
        self.zone = zone

    def getStatus(self) -> str:
        return "ZoneSensor[" + self.id + "]: zone=" + self.zone

    def emulate(self) -> None:
        self.zone = random.choice(self.ZONES)
```

```
    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {"id": self.id, "value": self.zone, "zone": self.zone}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        if request is None:
            return {"zone": self.zone}
        raw = request.args.get("value", "") or ""
        if raw:
            self.zone = raw
        return {"power": "on", "zone": self.zone}

class EnvSensor(Thing):
    def __init__(self, thing_id: str, name: str, temperature: float, light_lux: float) ->
None:
        super().__init__(thing_id, name)
        self.temperature = float(temperature)
        self.lightLux = float(light_lux)

    def getStatus(self) -> str:
        return (
            "EnvSensor["
            + self.id
            + "]: t="
            + str(round(self.temperature, 1))
            + ", lux="
            + str(int(self.lightLux))
        )
```

```
    def emulate(self) -> None:
        self.temperature = round(random.uniform(18.0, 28.0), 1)
        self.lightLux = float(random.randint(300, 600))

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {
            "id": self.id,
            "value": self.temperature,
            "temperature": self.temperature,
            "lightLux": int(self.lightLux),
        }

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        if request is None:
            return {"power": "on", "value": self.temperature}
        raw = request.args.get("value", "")
        if raw:
            self.temperature = round(float(raw), 1)
        return {"power": "on", "value": self.temperature}

class Scoreboard(Thing):
    def __init__(self, thing_id: str, name: str, score_a: int = 0, score_b: int = 0,
timer_sec: int = 0) -> None:
        super().__init__(thing_id, name)
        self.scoreA = score_a
        self.scoreB = score_b
        self.timerSec = timer_sec

    def getStatus(self) -> str:
```

```
    return (
        "Scoreboard["
        + self.id
        + "]: "
        + str(self.scoreA)
        + ":"
        + str(self.scoreB)
        + ", t="
        + str(self.timerSec)
    )

def emulate(self) -> None:
    self.timerSec += random.randint(0, 2)
    if random.random() < 0.3:
        self.scoreA += 1

def connect(self, request: Any = None) -> Dict[str, Any]:
    self.emulate()
    return {
        "id": self.id,
        "value": self.scoreA,
        "scoreA": self.scoreA,
        "scoreB": self.scoreB,
        "timer": self.timerSec,
    }

def connect_with_commands(self, request: Any) -> Dict[str, Any]:
    if request is None:
        return {"scoreA": self.scoreA, "scoreB": self.scoreB}
    raw = request.args.get("value", "")
```

```
        parts = raw.split("-")
        if len(parts) == 2:
            self.scoreA = int(parts[0])
            self.scoreB = int(parts[1])
        return {"power": "on", "scoreA": self.scoreA, "scoreB": self.scoreB}

class CourtDataStore:
    def __init__(self) -> None:
        self.events: List[str] = []
        self.telemetry: List[str] = []

    def saveEvent(self, data: Any) -> None:
        self.events.append("event=" + repr(data))

    def saveTelemetry(self, data: Any) -> None:
        self.telemetry.append("telemetry=" + repr(data))

    def getRecords(self) -> List[str]:
        return self.events + self.telemetry

class MainControlUnit:
        def __init__(self, things: List[Thing], store: CourtDataStore, scoreboard:
Scoreboard) -> None:
        self.things = things
        self.store = store
        self.scoreboard = scoreboard

    def collectData(self) -> None:
        for t in self.things:
            self.store.saveTelemetry({"id": t.id, "status": t.getStatus()})
```

```
def calculateScore(self) -> None:
    for t in self.things:
        if isinstance(t, ScoreSensor) and t.is_goal:
            self.scoreboard.scoreA += 1
            self.store.saveEvent({"type": "basket", "points": self.scoreboard.scoreA})
            break

def updateScoreboard(self) -> None:
    self.scoreboard.timerSec += 1
```